Public editions adapted from real Career Agent Pro outputs

Edição pública adaptada de uma saída real do produto. Material educacional gerado por IA e revisado para publicação. Todos os contextos de arquitetura, equipes, incidentes, requisitos e avaliação são hipóteses didáticas de plataformas fictícias. Não descrevem informações internas confirmadas, questões oficiais, contratação ou estratégia de empresas reais. Sem vínculo ou endosso de Prezensa, Globo, WhatsApp ou Meta.

Coding challenge Android: pagamentos e idempotência

Liderança Android e sistemas financeiros para plataforma fictícia de mensageria. Nível didático: sênior / liderança técnica. Este documento não descreve uma vaga real.

Visão Geral do Desafio

Cenário fictício: você lidera a modelagem de pagamentos numa plataforma de mensageria. Projete intenções de pagamento, estados auditáveis e recuperação de falhas em dispositivos com recursos limitados e rede instável; nenhum mandato de empresa real é descrito.

> **Atenção:** Você tem **45 minutos**. Priorize *correctness* antes de completude - uma solução parcial e bem fundamentada vale mais do que uma solução completa e frágil.

O Que Deve Ser Implementado

Implemente o núcleo de um **gerenciador de transações Pix offline-first** para Android.

O sistema deve ser capaz de:

1. Receber uma solicitação de pagamento do usuário
2. Tentar processar a transação via API remota
3. Persistir a intenção antes da primeira chamada de rede. Após falha ou resposta ambígua, preservar identidade e reconciliar estado; retentar apenas quando contrato, consentimento e TTL permitirem.
4. Permitir reenvios da mesma intenção com a mesma chave, sem duplicar efeitos financeiros, mediante deduplicação atômica no servidor simulado e reconciliação.
5. **Expor o estado da transação para a UI** de forma reativa

> **Dica estratégica:** Você **não precisa implementar a UI nem a API real**. O foco é na lógica de domínio, gerenciamento de estado e garantias de *correctness*.

Regras e Restrições

Categoria	Detalhe
Linguagem	Kotlin
Permitido	Coroutines, Flow, Room (esquema), WorkManager (conceitual)
Proibido	Bibliotecas externas de gestão de estado (Redux, MVI externo, etc.)
Compilação	Não é obrigatória - o código precisa ser **coerente e consistente**
Decisões de design	Devem ser **nomeadas e justificadas** em voz alta ou em comentário

Contrato de Entrada e Saída

Entrada

```
data class PixPaymentRequest(
    val idempotencyKey: String, // UUID gerado pelo cliente
    val recipientKey: String,   // chave Pix do destinatário
    val amountCents: Long,      // valor em centavos
    val description: String?
)
```

Saída esperada

Um `Flow` ou `StateFlow` que emite os seguintes estados:

```
sealed class TransactionState {
    object Idle : TransactionState()
    object Submitting : TransactionState()
    data class PendingRetry(val attemptCount: Int) : TransactionState()
    data class Reconciling(val intentId: String) : TransactionState()
    data class Success(val transactionId: String) : TransactionState()
    data class Failed(val reason: String, val retryable: Boolean) : TransactionState()
}
```

Rubrica didática da entrega

Não é esperado um sistema completo e compilável. O facilitador da simulação quer ver:

- [ ] **Estrutura de camadas** - como você organiza Repository, Domain e LocalDataSource
- [ ] Idempotência: manter identidade da intenção, reutilizar chave nos retries e especificar deduplicação atômica e consulta de resultado no servidor.

- [ ] **Lógica de retry** - quando retentar, quantas vezes e com qual estratégia
- [ ] **Contrato com a UI** - como o estado é exposto de forma reativa
- [ ] **Tradeoffs nomeados** - onde você tomou decisões e por quê

Edge Cases - O Que Será Explorado

Na simulação, use estes cenários como base para os *follow-ups*:

Cenário	Pergunta da simulação
A rede cai **após o servidor receber**, mas antes de responder	Como evitar que o usuário veja "falha" quando a transação foi processada?
O app é fechado com uma transação em `PendingRetry`	O WorkManager resolve? Como garantir a retentativa após reinício?
O usuário clica em "Pagar" duas vezes rapidamente	Quando e como o `idempotencyKey` é gerado? Quem é o dono dessa responsabilidade?
O servidor retorna HTTP 200 com corpo malformado	Isso é sucesso ou falha? O que é persistido?
A transação fica em `PendingRetry` por 48 horas	Existe TTL? Quem define o timeout? Como o usuário é informado?
Dois dispositivos do mesmo usuário tentam a mesma transação simultaneamente	É um problema do cliente ou do servidor? Qual é sua responsabilidade no Android?
No contrato simulado: HTTP 402 significa recusa por saldo; como distinguir recusa terminal de resultado incerto?	Você retenta? Por quê não? Como distinguir erros retentáveis de terminais?

> **Dica estratégica:** Você não precisa resolver todos esses casos no código. Mas precisa demonstrar que **sabe que eles existem** e que tem uma posição fundamentada sobre cada um.

Critérios didáticos de avaliação

Correctness - 40%

Critério	Peso
Idempotência implementada de forma confiável	Alto
Ausência de race conditions no submit + retry	Alto
Distinção clara entre erros retentáveis e terminais	Médio
Tratamento do estado "rede caiu durante o envio"	Médio

Design e Arquitetura - 30%

Critério	Peso
Separação de responsabilidades entre camadas	Alto
Contrato de API interno coeso e testável	Alto
Uso correto de `sealed class`, `data class` e `interface`	Médio
Uso adequado de Coroutines e escopos de lifecycle	Médio

Raciocínio e Tradeoffs - 20%

Critério	Peso
Capacidade de nomear o que não implementou e por quê	Alto
Consciência dos limites da solução proposta	Alto
Decisões de design explicadas, não apenas codificadas	Médio

Comunicação - 10%

Critério	Peso
Clareza ao explicar a solução sem ser perguntado	Médio
Capacidade de responder *follow-ups* sem defensividade	Médio

Exemplo de raciocínio de liderança técnica

> **Dica estratégica:** O facilitador da simulação não quer a solução perfeita. Ele quer ver **como você pensa sob pressão e incerteza**.

Antes de escrever qualquer linha de código

Reserve **3 a 5 minutos** para alinhar premissas em voz alta:

> Antes de enviar, crie e persista uma identidade estável para a intenção de pagamento. Reutilize a mesma idempotencyKey em retries, reinício e reabertura da mesma intenção. Só uma nova intenção explicitamente confirmada recebe nova chave. Deduplicação local evita trabalho redundante; impedir efeito financeiro duplicado exige contrato e atomicidade no servidor.

Ao escrever o repositório

Nomeie o tradeoff da estratégia de persistência:

> Persistir a intenção no Room antes da rede reduz perda de contexto após interrupção. Isso depende de commit local durável; não é garantia contra perda física de dados nem confirmação financeira. Após resposta perdida, reconciliar por identidade com o servidor antes de classificar resultado ou repetir a operação.

Ao implementar retry

Diferencie com precisão os tipos de erro:

> Contrato fictício: neste exercício, 402 significa recusa por saldo e não deve disparar retry automático. 409 pede consulta de resultado; não significa sucesso por si só. Timeout, perda de resposta ou falha 5xx podem ter efeito incerto: consultar/reconciliar e reutilizar a mesma chave antes de reenviar. Esses mapeamentos não são um padrão de Pix nem de API empresarial.

Ao ser questionado sobre app fechado durante retry

> KEEP é uma política de trabalho único: preserva trabalho pendente existente com o mesmo nome, não uma garantia de prazo ou de efeito financeiro único. Enfileire trabalho durável com identidade estável, constraints e backoff; a execução depende do SO e condições do dispositivo. Sessão ativa pode reconciliar imediatamente; background oferece recuperação adiada. Servidor permanece autoritativo e TTL/consentimento limitam reenvios.

Ao encerrar o tempo

Nomeie explicitamente o que ficou fora do escopo:

> *"Não implementei o TTL para transações presas em `PendingRetry`, nem a notificação ao usuário nesses casos. No contexto real, isso seria coordenado com o time de produto - há uma decisão de UX aqui que vai além do técnico. Minha prioridade foi garantir que a janela de correctness fosse sólida antes de qualquer feature de experiência."*

Sinais de Avaliação

Pontos de atenção

- [ ] Implementar retry sem qualquer mecanismo de idempotência
- [ ] Não distinguir erros retentáveis de terminais
- [ ] Não saber responder o que acontece quando a rede cai durante o envio
- [ ] Tratar todos os erros de forma genérica: `catch (e: Exception) { retry() }`
- [ ] Não nomear nenhum tradeoff durante os 45 minutos

Sinais de excelência

- [ ] Implementar write-local-first com estado auditável

- [ ] Nomear o problema do *“acknowledged but not responded”* proativamente
- [ ] Diferenciar a estratégia de retry em foreground vs. background
- [ ] Fazer ao menos uma pergunta de alinhamento antes de começar a codificar
- [ ] Encerrar o tempo declarando explicitamente o que ficou fora e por quê

Estado incerto e segurança do contrato

Reconciling representa resultado desconhecido após perda de resposta; não exibir Failed nem Success sem evidência autoritativa. PendingRetry indica elegibilidade para nova tentativa da mesma intenção, não autorização para criar outra chave. Success só após confirmação do servidor. Valores, moeda, destinatário e payload precisam ser validados no contrato simulado; recipientKey usa exclusivamente dados sintéticos.

Este é um contrato de demonstração, não um SDK de pagamento pronto para produção. Inclua nos testes: dupla ação concorrente, restart entre commit local e envio, resposta perdida após commit do servidor, reutilização de chave com payload divergente e expiração da janela de deduplicação.

Contrato de confiabilidade do cenário financeiro

O exercício usa um servidor simulado autoritativo. Uma intenção tem identificador e idempotencyKey estáveis, associados à identidade autenticada e ao payload; reutilizar a chave com outro payload deve ser rejeitado. Deduplicação e registro do efeito precisam ser atômicos no servidor, com retenção de chaves compatível com a janela de retries. O cliente, sozinho, não garante processamento único.

Persistir intenção antes da rede reduz perda de contexto, mas não confirma liquidação. Timeout pode ocorrer após execução: manter estado incerto, consultar por identidade e reconciliar transições monotônicas. Nova tentativa da mesma intenção reutiliza a chave; nova intenção exige confirmação. Validar expiração, cancelamento e autorização antes de retomar trabalho antigo. Histórico offline não autoriza confirmar ou executar pagamento offline.

Notificações são pistas de atualização, não a fonte da verdade. Deduplicar eventos por identificador/versão e buscar estado ao abrir o app e quando houver recuperação permitida. Logs e eventos de demonstração devem omitir chaves Pix reais, valores pessoais e tokens; usar dados sintéticos. Persistência local e retenção exigem minimização, limpeza por conta e proteção adequada; não implicam conformidade legal automática.

Referências técnicas e escopo

Referências de APIs públicas para estudo; não são evidência sobre arquitetura ou processos de empresas reais. Os contratos Kotlin são exemplos educacionais, sem implementação de produtos privados.

- Android: coroutines e ciclo de vida
- Android: WorkRequests e constraints
- Firebase: prioridade de mensagens Android

Career Agent PRO

Edição pública adaptada de saída real. Profundidade técnica, hipóteses explícitas e decisões verificáveis.

www.career-agent-pro.com

support@main.career-agent-pro.com

© 2026 Career Agent PRO. Todos os direitos reservados.

ADAPTED PUBLIC EDITION